

Conditional Jacobian Aggregation

Optimizing GPU Track Finding through Algorithmic Refinement

Hao-Chun, Liang

GPU Optimization for ACTS/traccc

January 2026

Advisor: Prof. Bo-Cheng, Lai

Outline

- 1 Problem Context
- 2 Implementation
- 3 Results
- 4 Lessons Learned
- 5 Conclusion

Register Pressure in GPU Track Finding

The Problem

- propagate_to_next_surface kernel
- Only **10-25% GPU occupancy**
- Root cause: High register usage (128-203 registers/thread)
- V100 needs ≤ 32 registers for 100% occupancy

Impact

- SM underutilization
- Poor latency hiding
- Reduced throughput

Register Budget Analysis

Component	Registers
Bound parameters (6 floats)	6
Covariance (6×6 symmetric)	21
Jacobian (6×6)	~ 36
RK4 derivatives	12
B-field vectors	9
Navigation state	8-12
Actor states	20-30
Total	120-150

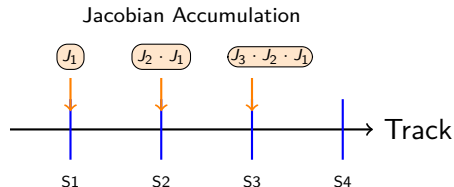
Key Insight

Jacobian is only needed for MBF smoother!

Opportunity: Conditional Jacobian

When is Jacobian Needed?

- Multi-Branch Fit (MBF) smoother requires accumulated Jacobian
- Used for backward smoothing pass
- When MBF disabled: Jacobian computed but **never used**



Current Behavior (parameter_transporter)

- 1 Compute full 6×6 Jacobian at each surface
- 2 Multiply: $J_{acc} = J_{new} \times J_{acc}$ ($6 \times 6 \times 6 \times 6$)
- 3 Store accumulated Jacobian to global memory
- 4 **Wasted work when MBF=false!**

Savings per Surface

- ~ 216 FLOPs (matrix mult)
- ~ 288 bytes memory I/O

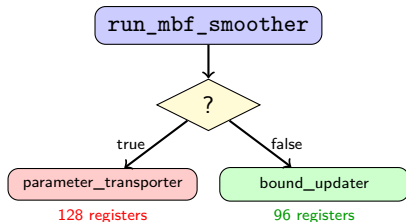
Solution: Two Actor Chains

Original: ckf_actor_chain_t

- 1 pathlimit_aborter
- 2 **parameter_transporter** ← Has Jacobian ptr
- 3 interaction_register
- 4 ckf_interactor
- 5 parameter_resetter
- 6 momentum_aborter
- 7 ckf_aborter

New: ckf_actor_chain_no_mbf_t

- 1 pathlimit_aborter
- 2 **bound_updater** ← Empty state!
- 3 interaction_register
- 4 ckf_interactor
- 5 parameter_resetter
- 6 momentum_aborter



Compile-Time Dispatch

if constexpr selects actor chain based on propagator type

Key Code: Actor State Comparison

parameter_transporter (detray)

```
// State contains Jacobian pointer
struct state {
    bound_matrix_t* _full_jacobian_ptr
        = nullptr;
};

// In operator():
if (actor_state._full_jacobian_ptr
    != nullptr) {
    // 6x6 x 6x6 matrix multiplication
    const auto aggregate =
        full_jacobian *
        (*(actor_state._full_jacobian_ptr));
    // Write back to global memory
    (*(actor_state._full_jacobian_ptr)) =
        aggregate;
}
```

bound_updater (traccc)

```
// Empty state - no Jacobian pointer!
struct state {};

// In operator():
// Compute full_jacobian (same as before)
const auto full_jacobian =
    get_full_jacobian(propagation);

// Transport covariance (same as before)
transport_covariance_to_bound_impl(
    old_cov, full_jacobian, new_cov);

// NOTE: No Jacobian aggregation!
// This is the key difference.
```

Key Difference

bound_updater computes Jacobian for covariance transport but **skips the 6×6 matrix multiplication and global memory write.**

propagate_to_next_surface.ipp

```
// Compile-time check for actor chain type
if constexpr (
    details::has_jacobian_transport_v<
        propagator_t>) {
    // 7-actor chain with param_transporter
    // Initialize Jacobian for MBF smoother
    payload.tmp_jacobian_ptr[param_id] =
        matrix::identity<bound_matrix_t>();
    s1._full_jacobian_ptr =
        &payload.tmp_jacobian_ptr[param_id];

    propagator.propagate(propagation,
        detrayer::tie(s0,s1,s2,s3,s4,s5,s6));
} else {
    // 7-actor chain with bound_updater
    // No Jacobian initialization needed

    propagator.propagate(propagation,
        detrayer::tie(s0,s1,s2,s3,s4,s5,s6));
}
```

Type Trait Detection

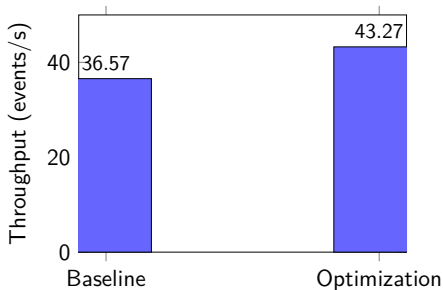
```
/// Check if propagator uses Jacobian
template <typename propagator_t>
struct has_jacobian_transport
    : contains_type<
        parameter_transporter<algebra_t>,
        typename propagator_t::
            actor_chain_type::actor_tuple
    > {};

template <typename propagator_t>
inline constexpr bool
has_jacobian_transport_v =
has_jacobian_transport<
    propagator_t>::value;
```

Kernel Specializations

- 3 detectors $\times$ 3 B-fields $\times$ 2 MBF
- = 18 kernel variants

Benchmark Results



Apples-to-apples: Both with MBF=false

Performance Summary

Metric	Value	Change
Throughput	43.27 ev/s	+18.3%
Latency	23.11 ms	-15.5%

Test Results

710/710 CUDA tests passing

Physics correctness validated

Configuration

- Dataset: ttbar_mu200
- Baseline: a48cc783
- Optimization: 25894cca
- V100-SXM2-32GB, 8 CPU threads

Methodology: Isolating the True Improvement

Initial Result: +11.67%

- First benchmark showed 38.75 $\rightarrow$ 43.27 ev/s
- But build_tracks improved by **-86.3%**!
- Something else was happening...

Discovery: Configuration Change

- Baseline: `run_mbf_smoother = true`
- Optimization: `run_mbf_smoother = false`
- This was a **default value change**, not just the actor swap

Confounded Effects

Change	Impact
MBF default true $\rightarrow$ false	-86% build_tracks
Jacobian aggregation skip	+18.3% throughput

Apples-to-Apples Re-benchmark

- Both commits with MBF=false
- Baseline: 36.57 ev/s (not 38.75)
- True improvement: **+18.3%**

Lesson

Always control for configuration changes!

NCU Profiling: Register Reduction Confirmed

RTX 2080 Ti (sm_75), ncu 2024.1.1, single event, 1 CPU thread

Register & Occupancy Analysis

Metric	Base	Opt	Δ
Registers/thread	128	96	-25%
Block limit (regs)	4	5	+1
Theor. occupancy	50%	62.5%	+12.5%
Achieved occupancy	39.3%	48.6%	+9.3%
Active warps/SM	12.6	15.6	+3.0

Key Finding

32 registers saved (128 $\rightarrow$ 96)

Enables 1 additional block per SM

Compute & Memory Analysis

Metric	Base	Opt	Δ
Duration (ms)	3.99	3.61	-9.5%
Instructions	82.3M	78.5M	-4.6%
Mem throughput	218 GB/s	245 GB/s	+12.1%

Dual Optimization Mechanism

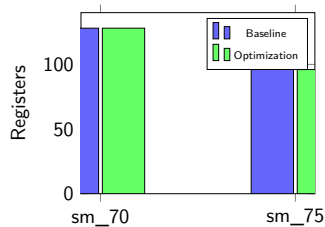
- 1 **Register reduction** $\rightarrow$ Higher occupancy
- 2 **Fewer instructions** $\rightarrow$ Less work

Both contribute to 9.5% kernel speedup

Architecture-Dependent Behavior

Profiling Tools Comparison

Tool	GPU	Base	Opt
cuobjdump	sm_70 (V100)	128	128
ncu	sm_75 (2080 Ti)	128	96



Why the Difference?

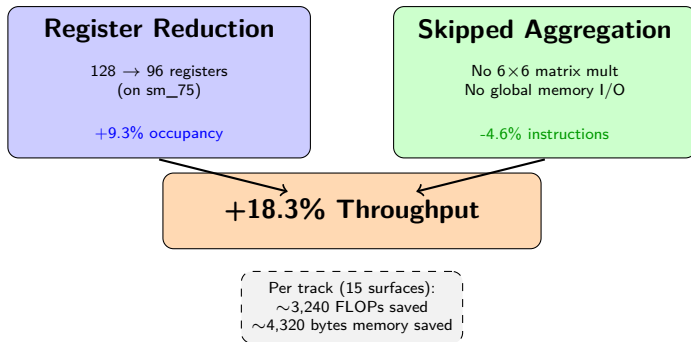
- Different GPU architectures compile differently
- Compiler makes different register allocation decisions
- sm_75 has additional instructions

Implication

Always profile on target hardware!

Register benefits are architecture-dependent

Two Complementary Mechanisms



Key Takeaways

What We Learned

① Profile on target hardware

- cuobjdump (sm_70): No register reduction
- ncu (sm_75): 25% register reduction

② Original claims were partially validated

- Expected: ~ 36 register savings (6×6 matrix)
- Actual: 32 registers (close! but only on sm_75)

③ Multiple mechanisms contribute

- Register reduction
- Instruction reduction
- Memory traffic reduction

Post-Mortem: Claim vs Reality

Claim	Expected	Actual
Register reduction	-36 regs	-32 (sm_75 only)
Occupancy gain	+10-25%	+9.34% (sm_75)
Throughput	+5-15%	+18.3%

Important Caveat

Optimization only works when **MBF smoother is disabled**.

Trade-off: No backward smoothing pass.

Summary

Achievements

- **+18.3%** throughput improvement
- **-25%** register usage (on sm_75)
- **+9.3%** achieved occupancy
- **710/710** tests passing

Implementation

- New `bound_updater` actor
- Two actor chain variants
- 18 kernel specializations
- Compile-time dispatch

Mechanism

- Skip 6×6 matrix multiplication
- Reduce global memory traffic
- Architecture-dependent register savings

Key Files

- `bound_updater.hpp`
- `combinatorial_kalman_filter_types.hpp`
- `propagate_to_next_surface.ipp`

Conclusion

Conditional Jacobian Aggregation provides **+18.3% throughput** through dual mechanism: register reduction (architecture-dependent) and skipped matrix operations (universal).

Thank you for your attention!

`github.com/acts-project/traccc`

Documentation: `doc/conditional_jacobian_transport_*.md`

Backup: Anticipated Questions (1/2)

Q1: Why does Jacobian computation still happen?

- `get_full_jacobian()` is needed for covariance transport
- We skip the *aggregation*, not the computation
- Future work: skip computation entirely when not needed

Q2: Why not runtime dispatch instead of compile-time?

- `if constexpr` allows dead code elimination
- Runtime `if` would still allocate registers for Jacobian pointer
- Compile-time dispatch enables different kernel specializations

Q3: What is MBF smoother and when is it needed?

- Multi-Branch Fit smoother: backward pass to improve track parameters
- Needed for high-precision physics measurements
- Can be disabled for real-time trigger applications

Backup: Anticipated Questions (2/2)

Q4: Why different results on sm_70 vs sm_75?

- CUDA compiler makes architecture-specific optimizations
- Different instruction sets (sm_75 has tensor cores)
- Register allocation heuristics vary by architecture
- Always profile on production hardware

Q5: What about other register-heavy kernels?

- fit_forward: 168 registers
- fit_backward: 168 registers
- Similar conditional optimization could apply
- Future work opportunity

Q6: How does this compare to other optimizations?

- Multi-event batching: +93% throughput
- Conditional Jacobian: +18.3% throughput
- Complementary optimizations, can be combined

Backup: Detailed NCU Metrics

Full Profiling Data (RTX 2080 Ti, sm_75, ncu 2024.1.1)

Metric	Base	Opt	Δ
<i>Occupancy</i>			
Registers/thread	128	96	-25%
Block limit	4	5	+25%
Theor. occupancy	50.0%	62.5%	+12.5%
Achieved occupancy	39.28%	48.62%	+9.34%
<i>Compute</i>			
Duration (ms)	3.99	3.61	-9.5%
Instructions (M)	82.3	78.5	-4.6%
IPC	0.26	0.26	0%

Metric	Base	Opt	Δ
<i>Memory</i>			
Mem throughput	218 GB/s	245 GB/s	+12%
DRAM throughput	33.2%	37.7%	+4.5%
L1 hit rate	54.1%	46.3%	-7.8%
L2 hit rate	79.6%	76.6%	-3.0%
<i>Scheduler</i>			
No eligible (%)	93.24%	93.27%	+0.03%
Active warps/sched	3.23	4.03	+0.80
Warp cycles/instr	47.74	59.92	+25%

Bottleneck Analysis

Kernel remains **latency-bound**: 93% of cycles have no eligible warps. Note: Warp cycles/instruction *increased* despite speedup—fewer total instructions but each waits longer (more memory parallelism).